
Smart Warehouse IoT System using Autonomous Mobile Robot (AMR)

מערכת מחסן חכם באמצעות רובוט נייד אוטונומי (AMR)

Adi Shlomo (211619226), Shahar Halevy (209058114)
עדִי שלמה (211619226), שחר הלוי (209058114)

Instructor: Dr. Guy Tel-Tzur
מנחה הקורס: ד"ר גיא תל-צור

GitHub: [ShaharHalevi/ESP32-MicroROS-AWS-Bot \(main\)](#)

Demo (YouTube): [Project Demo Video](#)

Abstract

Smart warehouse inventory tracking is very important, but many solutions are too expensive for small businesses. This paper presents a cost-effective system for automated inventory updates using an Autonomous Mobile Robot (AMR). The system is built on the ESP32 microcontroller. To control the robot's movement efficiently, we utilized the micro-ROS framework, which allows direct communication with the Robot Operating System (ROS 2). For identifying products, an ESP32-CAM module scans QR codes. The inventory data is transmitted to the cloud using the lightweight MQTT protocol through a local gateway. The cloud architecture uses Amazon Web Services (AWS): messages are received by AWS IoT Core, processed by serverless Lambda functions, and stored in DynamoDB. A Grafana dashboard shows the stock levels in real-time, and email alerts are sent if the inventory is low. The

system ensures secure communication and is designed for scalability. Future work includes adding AI for hazard detection. The results demonstrate that it is possible to build a reliable automation solution using low-cost hardware and standard IoT protocols.

1 Introduction

Warehouses are a crucial part of the supply chain system. When inventory data is not accurate or delayed, it causes serious issues such as stock-outs or excess inventory. The main goal of Smart Warehouse technology is to change the traditional warehouse into a data-driven system. By collecting data in real-time from the floor, the system allows us to manage the stock proactively instead of reacting to problems after they occur. (Yerpude and Singhal, 2018)

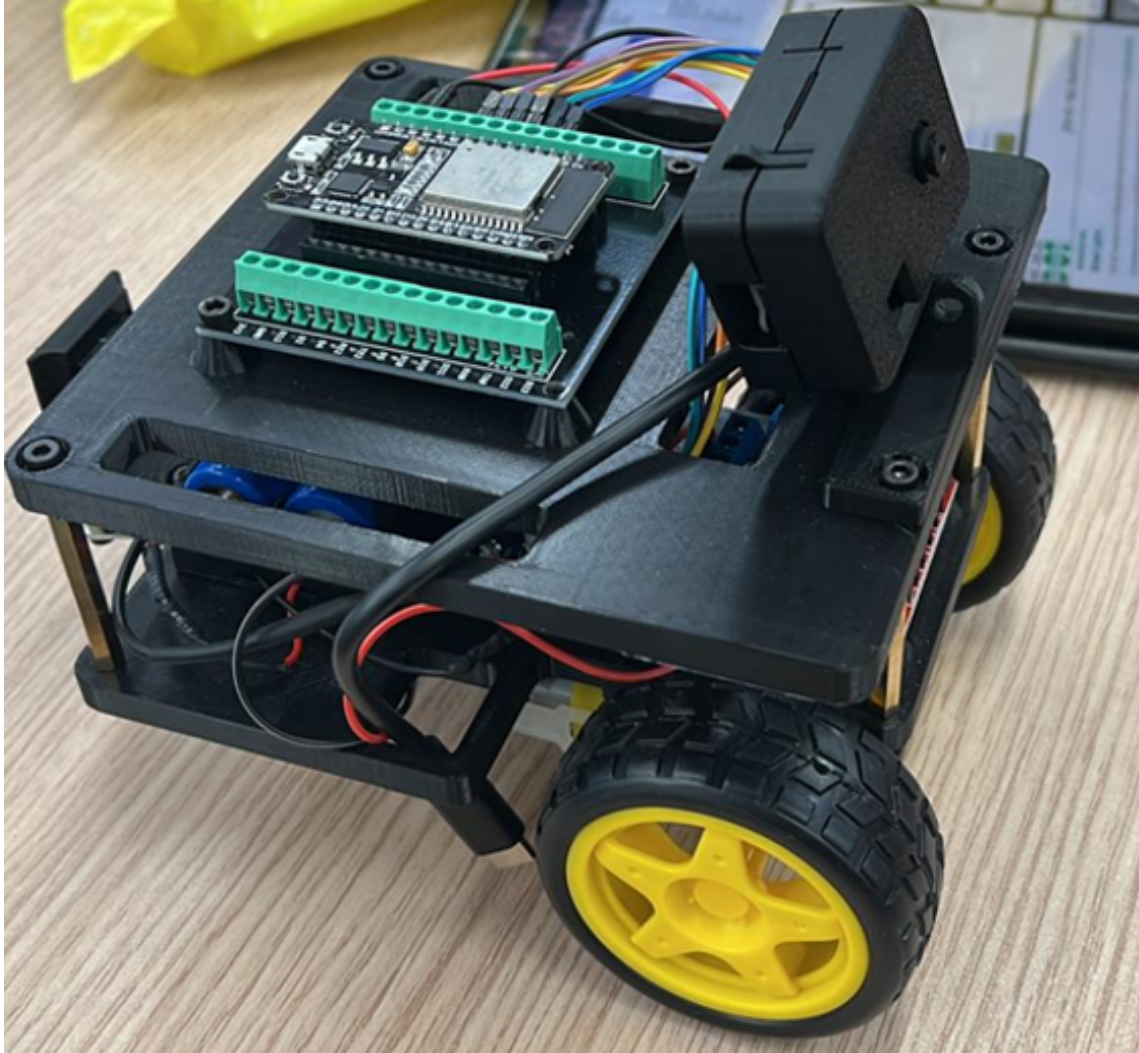


Figure 1: Prototype AMR platform used in this project. The top deck hosts the ESP32-based controllers and power distribution, while the front-mounted camera module is positioned to maintain line-of-sight to box-mounted QR codes during navigation.

1.1 Market survey and motivation

Several mainstream approaches are used today to improve warehouse visibility. *Smart shelves* install sensors on every single shelf to enable continuous monitoring, but they often require dense deployments and high infrastructure and wiring costs, making them challenging to adopt at small scale. (Chai et al., 2023) *RFID-based tagging* enables automatic identification and tracking. However, this method requires installing readers everywhere and putting tags on every single item; for low-cost products, the per-item tagging effort and ongoing operational overhead can become a barrier. (Chai et al., 2023; Sagaya Selvaraj and Anusha, 2021) At the other end of the spectrum, *industrial robotics and AI-driven fulfillment systems* demonstrate major efficiency gains,

but they are usually designed for enterprise-scale operations and budgets, creating a high entry barrier for small and medium-sized businesses. (Li, 2024) As a result, there is a practical gap: many small warehouses need an affordable solution that provides full coverage and near-real-time updates without requiring shelf retrofitting, item-level RFID operations, or enterprise-scale robotics.

1.2 Problem statement and requirements

The goal of this project is to design and implement a cost-effective system that can (1) scan products automatically, (2) update inventory in the cloud in near real time, and (3) provide mon-

itoring and alerts for low-stock events. In addition to cost, two non-functional requirements are critical for real deployments: **reliability** under intermittent connectivity (to avoid data loss and ensure recovery), and **security** for device-to-cloud communication.

1.3 Our approach

We propose an IoT-based inventory tracking system built around an Autonomous Mobile Robot (AMR) that scans existing QR codes on boxes, requiring no changes to the products themselves. The system follows a *hybrid architecture* that separates the control path from the data path: real-time robot control is kept independent from cloud-bound data handling to maintain stable motion under variable network conditions. To reduce latency sensitivity compared to WiFi-based control loops, the robot control uses Micro-ROS over UDP, and processing load is reduced by splitting responsibilities across dedicated controllers (motion vs. camera/QR decoding). For inventory data, scan events are sent via MQTT to a local gateway that bridges the warehouse network to cloud services. The data flow in the cloud is designed as a pipeline. Messages arrive at AWS IoT Core and trigger an AWS Lambda function. This function checks the inventory logic and saves the results to Amazon DynamoDB. For monitoring, we connected a Grafana dashboard to the database. Additionally, Amazon SNS is used to send automatic notifications when the system detects a shortage.

1.4 Reliability, security, and scalability

To handle network instability, the system includes automatic reconnection logic that restores communication if the connection drops. For security, we utilized mutual authentication with X.509 certificates and TLS 1.2 encryption, ensuring that only authorized devices can connect to AWS IoT Core. Finally, the architecture is designed to scale. Since the cloud functions are stateless and we use specific MQTT topics, it is easy to extend the system from a single robot to a fleet of robots.

1.5 Contributions

The main contributions of this work are: (1) a low-cost AMR workflow for QR-based inventory updates without shelf retrofitting or item-level RFID operations, (2) a hybrid control/data architecture that maintains real-time responsiveness while leveraging cloud services, (3) an end-to-end cloud flow with monitoring and alerts, including reliability (buffering/recovery) and security (TLS/X.509), and (4) a scalability path toward multi-robot warehouse coverage.

1.6 Paper organization

The remainder of this paper is organized as follows. Section 2 presents the methodology and the design choices for the AMR-based scanning workflow. Section 3 describes the end-to-end system architecture, including the local gateway and the AWS cloud pipeline. Section 4 details the implementation and the experimental evaluation of the prototype. Finally, Section 5 concludes the paper and outlines limitations and future work.

2 Methodology

This project implements a cost-effective Smart Warehouse inventory tracking system based on an Autonomous Mobile Robot (AMR) and a hybrid edge cloud architecture. The methodology was designed around a strict separation between two pipelines: (1) a **control path** for real-time robot motion and navigation, and (2) a **data path** for inventory events (QR scans) and cloud synchronization. The full end-to-end flow is summarized in Fig. 4, from on-robot sensing and control, through the local gateway, and into the AWS cloud services for processing, storage, visualization, and alerting.

2.1 Mechanical platform and scanning viewpoint

The AMR platform is a differential-drive chassis with **two DC motors** and a rear **caster wheel** (Fig. 2). Differential drive enables forward/backward motion, smooth steering by varying wheel speeds, and **in-place rotation** when the wheels spin in opposite directions, which is useful for maneuvering in narrow warehouse aisles. The caster stabilizes the chassis while keeping the mechanical design simple. The upper deck is intentionally elevated to improve the camera line-of-sight to box-mounted QR codes, increasing the probability of successful scans while the robot is moving.

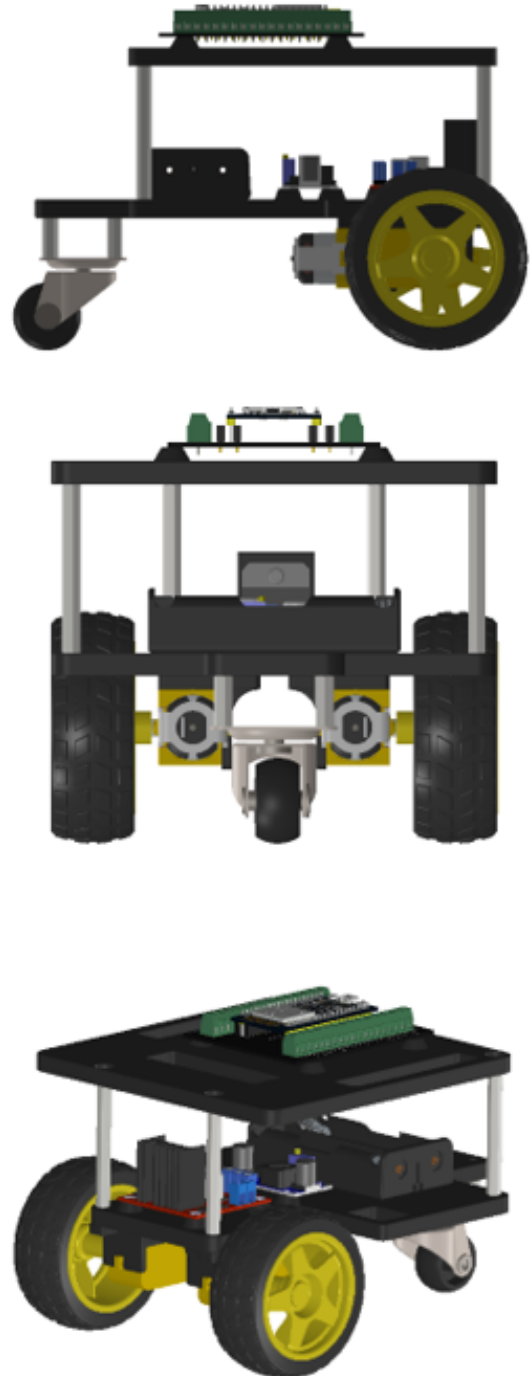


Figure 2: AMR mechanical platform (side/front/isometric views). Differential-drive with two DC motors enables in-place turns, while a rear caster stabilizes the chassis. The elevated top deck improves camera line-of-sight to box-mounted QR codes.

2.2 Hardware design and controller split

To preserve real-time motion behavior while supporting vision-based scanning, the robot electronics are split into two dedicated microcontrollers. An **ESP32 Motor Controller** executes the motion control loop and converts steering/speed commands into motor driver signals. It interfaces with an **L298N** H-bridge driver to control the two DC motors using control and PWM lines, as shown in the wiring schematic (Fig. 3). In parallel, an **ESP32-CAM** performs image acquisition and on-device QR decoding. This split prevents QR decoding from competing with the control loop for CPU time and ensures that scanning load does not degrade motion stability. Power distribution is handled by a battery supply regulated with a **buck converter** to provide stable voltage rails for the controllers and driver (Fig. 3).

2.3 Communication protocols (control vs. data)

Two communication stacks are used, each matched to the requirements of its pipeline. For the **control path**, we use **Micro-ROS over UDP** to minimize latency and overhead, enabling responsive steering updates and smoother teleoperation/navigation compared to heavier transport alternatives. For the **data path**, inventory events (decoded QR payloads) are published using **MQTT**, a lightweight pub/sub protocol widely adopted in IoT systems. MQTT supports efficient small messages and topic-based separation, which becomes important when scaling to multiple robots or multiple event types.

2.4 Local gateway station and buffering for reliability

A local **gateway station** bridges the robot network to the cloud (Fig. 4). On the gateway, a **Mosquitto** broker receives MQTT events from the robot, while a dedicated **Python AWS bridge** validates and forwards inventory events to AWS. The gateway also runs the **Micro-ROS**

agent that interfaces with the robot control nodes. To simplify deployment and avoid dependency issues, gateway services are containerized (e.g., running the Micro-ROS agent in Docker). Reliability under intermittent connectivity is addressed by a **buffering and recovery mechanism**: if the cloud connection drops, the gateway stores scan events locally rather than dropping them, and automatically re-synchronizes buffered events once connectivity is restored. This “store-and-forward” behavior prevents inventory data loss and allows continuous robot operation during temporary network outages.

2.5 Cloud pipeline and user interface

The cloud pipeline follows an ingestion processing storage model (Fig. 4). The gateway publishes events to **AWS IoT Core**, which serves as the managed MQTT entry point. Each message triggers **AWS Lambda** (serverless compute) to execute the inventory logic (e.g., update item quantities, apply thresholds). The system stores inventory records in **Amazon DynamoDB** to support fast queries and near real-time reads by the dashboard. A **Grafana** dashboard provides visualization of stock levels and scan history, and **Amazon SNS** triggers notifications (email/SMS) when inventory falls below a configured threshold.

2.6 Security and scalability

Security is enforced end-to-end through encrypted communication and authentication at the IoT entry point using **TLS 1.2** and **X.509 certificates**, combined with least-privilege permissions (IAM) and audit logging (CloudTrail) on the cloud side. Finally, the architecture supports scalability by design: MQTT topic separation, a gateway-based bridge, and stateless event-driven processing in the cloud enable future expansion from a single robot to a multi-robot fleet without changing the fundamental flow (Fig. 4).

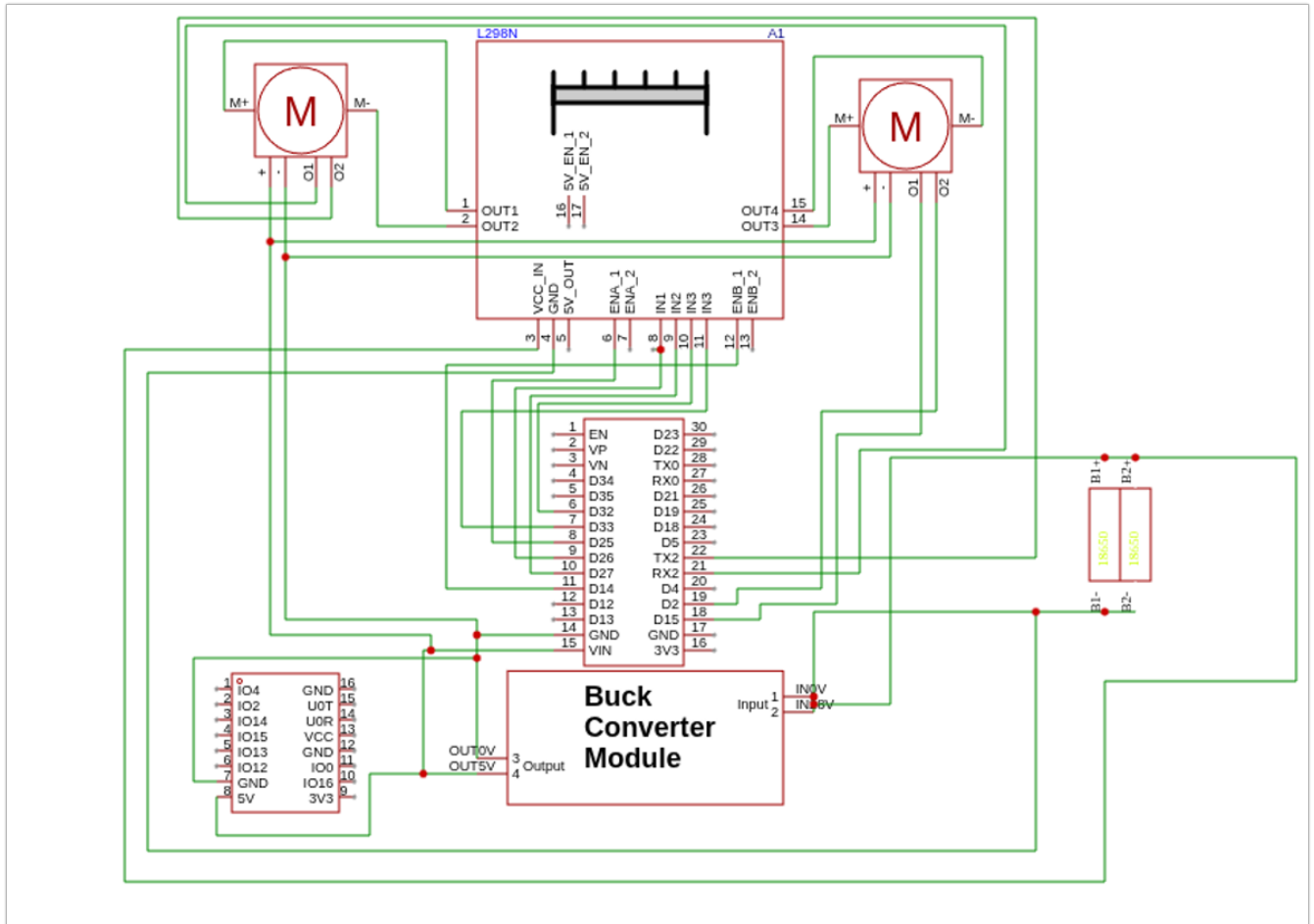


Figure 3: Robot hardware wiring schematic. The ESP32 motor controller drives the L298N motor driver (two DC motors) using control/PWM lines. A buck converter regulates the supply rails for the controllers and driver. The ESP32-CAM operates as an independent QR scanning node and publishes decoded events to the gateway.

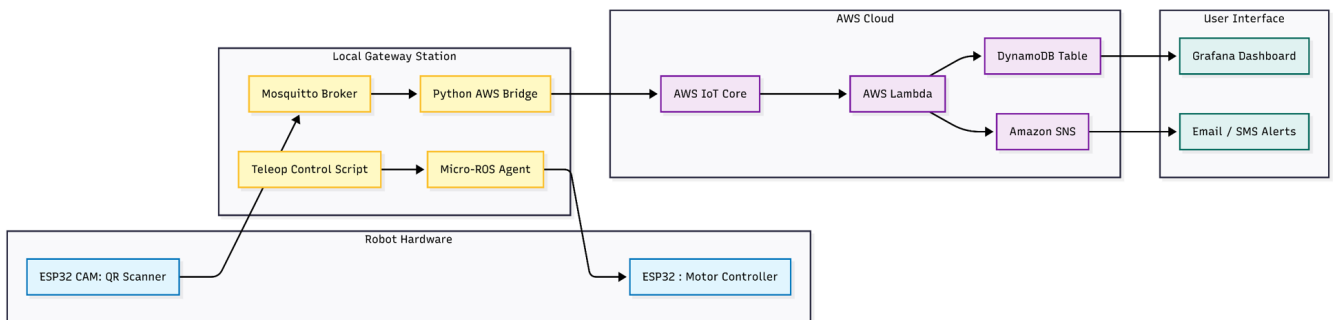


Figure 4: End-to-end system flow. The robot publishes QR scan events to a local gateway (Mosquitto broker + Python AWS bridge), while motion control is handled via Micro-ROS through the gateway agent. The gateway forwards inventory events to AWS IoT Core, triggers AWS Lambda for processing, stores records in DynamoDB, and exposes data to a Grafana dashboard with SNS-based email/SMS alerts.

3 Results

The system was evaluated using two complementary demonstrations that validate both end-to-end functionality and robustness under increased event volume. In the **live end-to-end demonstration**, the AMR scanned QR-coded products while the complete pipeline was monitored simultaneously at three layers: the local gateway, the AWS cloud services, and the user interface. Fig. 5 shows the experimental setup, where the robot performs scanning while the operator observes the system in real time. Each successful scan generated an inventory event that propagated from the robot to the local gateway (MQTT), was forwarded to AWS IoT Core, processed by AWS Lambda, and stored in DynamoDB. The updated stock state was then reflected in the Grafana dashboard, demonstrating the correctness of the ingestion–processing–storage–visualization chain.

To validate continuous **real-time visualization**, we monitored dashboard panels during repeated scan events. The time-series panel (Fig. 6a) shows how counts evolve over time for multiple product types, confirming consistent cloud updates without missing or sporadic data. In addition, the bar-chart snapshot (Fig. 6b) provides a clear view of the current inventory distribution across products, which is the expected operational interface for warehouse monitoring.

We further verified the **alerting mechanism** by triggering a low-stock condition. When the inventory count crossed the configured threshold, an automated AWS SNS notification was generated and delivered as an email alert (Fig. 7). This result validates not only data logging but also the system’s ability to actively notify operators in real time.

In a second experiment, we evaluated **scalability at the application level** using a Python-based simulation script that emulates a robot scanning a larger set of products at a higher rate. Instead of relying on manual scanning, the script generated a stream of synthetic scan events to stress the gateway-to-cloud pipeline. The system continued to behave correctly: events were ingested automatically, dashboard values updated consistently, and the same threshold-based alert logic remained functional. The complete live

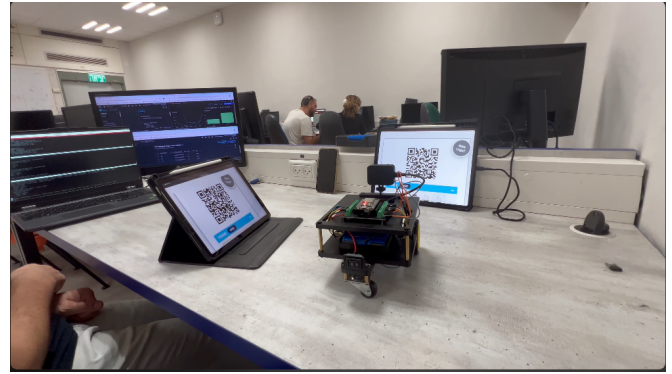
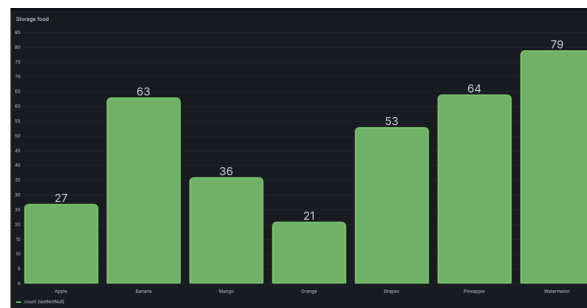


Figure 5: Live end-to-end demonstration setup. The AMR scans QR-coded products while the local gateway and the cloud dashboard are monitored in real time.

demonstration is provided in the project video linked at the beginning of the paper, and the full implementation is available in the project GitHub repository.



(a) Grafana time-series panel showing real-time inventory count updates for multiple product types during scanning events.



(b) Grafana snapshot view of current inventory distribution across products after processing scan events in the cloud.

Figure 6: Cloud dashboard outputs observed during evaluation.

⋮ → 😊 ☆ ((2) לופני יומיים) 17:52, 6 בינ', 6 ג' 17:52

<no-reply@sns.amazonaws.com> **AWS Notifications** ▼ 17:52

ALERT: Low stock detected!
Item: Apple
Box ID: 13
Quantity Remaining: 15

Figure 7: Automated low-stock alert delivered via AWS SNS email notification, triggered when an item count drops below the configured threshold.

4 Discussion

This project was intentionally built as a **proof-of-concept** that demonstrates an end-to-end Smart Warehouse workflow from physical scanning on an AMR, through a local gateway, and into a cloud pipeline with visualization and alerting rather than a fully performance-optimized industrial deployment. The main contribution is therefore the **system integration and operational feasibility**: showing that the complete flow works reliably, is easy to operate, and can be extended in a structured way.

4.1 Why QR/barcodes (SKU modeling and practicality)

A key design choice was using **QR/barcodes** as the product identifier. This matches common retail and warehouse practice where each product is associated with a unique **SKU** and

can be represented by a compact code printed on a label. Compared to alternatives such as RFID tagging or dense smart-shelf instrumentation, QR/barcodes are low-cost, simple to generate, and easy to integrate into a prototype with minimal infrastructure changes. In our context, they also provide a clean abstraction for system validation: a successful scan maps deterministically to a product ID, enabling consistent end-to-end testing of ingestion, processing, storage, and user-facing outputs.

4.2 What the live demonstration validates

The live demonstration (Fig. 5) validates the project's primary goal: **operational end-to-end automation**. Specifically, it shows that a scan event created at the robot is propagated through the local gateway and reflected in the cloud in near real time, while the operator can observe the full process through the dashboard interface (Figs. 6a and 6b). The real-time **SNS email alert** (Fig. 7) is particularly important in a warehouse setting because it demonstrates that the system is not only passive (logging data), but also **actionable**: it can notify an operator immediately when a threshold condition occurs (e.g., low stock). In practice, this is one of the main reasons to adopt smart inventory tools to reduce the time between a shortage emerging and a human taking corrective action.

4.3 Real-time expectation vs. evaluation focus

While the architecture is designed to support immediate updates (event-driven cloud processing and a lightweight telemetry protocol), the evaluation emphasis in this work is **correctness and usability** rather than measuring strict performance KPIs such as latency distributions or maximum throughput. In other words, we focused on proving that the system behaves as expected, that the user experience is clear (dashboard views and alerts), and that the operational workflow is practical for small-to-medium deployments. A natural next step for a more engineering-oriented study would be

adding quantitative measurements (end-to-end delay, event loss rate, and sustained events-per-second capacity).

4.4 Scalability demonstration and what it means

To demonstrate the system's scalability path, we introduced a second experiment in which a **Python-based scan simulator** generates scan events at higher volume and across a larger product set. This is not a substitute for a full multi-robot stress test, but it is a useful validation of the **cloud-side scale mode**: the ingestion pipeline (IoT Core), the stateless processing (Lambda), and the database updates (DynamoDB) continue to function correctly as event volume increases, without requiring design changes. Practically, this supports the claim that the system can grow from a small demo (two products) to a larger catalog, and later to multiple robots, by extending topic-based separation and device provisioning on the same architectural backbone.

4.5 Limitations and future work

The results also highlight realistic limitations typical for vision-based inventory identification. QR scanning requires adequate **line-of-sight**, sufficient lighting, and stable camera positioning; motion blur and extreme angles can degrade decoding reliability. In addition, the current evaluation includes a live demo and a software-based scale simulation, but does not yet include long-duration tests over days, diverse network conditions, or concurrent operation of multiple robots in the same area. Future work should therefore focus on: (1) improving scan robustness (camera placement refinement, exposure control, brief pause-on-scan policy, or improved decoding pipeline), (2) hardening system behavior under network variability (explicit retries, idempotent event IDs, and deduplication to prevent double-counting), (3) adding quantitative evaluation (latency/throughput/uptime), and (4) extending to multi-robot deployments with automated device onboarding and per-robot topic isolation.

4.6 Takeaway

Overall, the project demonstrates that a low-cost AMR combined with a gateway and managed cloud services can deliver a practical smart-inventory workflow with real-time monitoring and alerting. The proof of concept successfully highlights the operational advantages simple identification via SKU-coded labels, intuitive monitoring through a dashboard, and immediate alerts while establishing a clear roadmap for scaling and performance-oriented improvement.

5 Conclusion

This project presented a Smart Warehouse IoT system that combines a low-cost Autonomous Mobile Robot (AMR) with a cloud-connected inventory pipeline. The core objective was to demonstrate a practical end-to-end workflow for automated inventory tracking: the robot scans QR-coded products (SKU-based identification), generates structured inventory events, and synchronizes stock levels to the cloud where operators can monitor the system in real time and receive alerts when inventory drops below a threshold. Rather than targeting industrial-grade performance optimization, the project was designed as a proof-of-concept that highlights feasibility, integration quality, and operational value.

At the system level, the main contribution is the successful integration of all components required for a complete IoT solution. On the robot side, two ESP32-based nodes were used to separate responsibilities: an ESP32-CAM acts as an independent QR scanning unit, while an ESP32 motor controller handles motion and low-level control. This separation supports modularity and reduces coupling between perception and actuation. Communication over WiFi was designed with two distinct paths that reflect real operational needs: (1) real-time robot control and teleoperation using Micro-ROS over UDP, prioritizing low latency and responsiveness; and (2) reliable delivery of inventory events using MQTT through a local gateway. This dual-protocol approach is a practical trade-off: it preserves agility in control while keeping inventory telemetry structured and cloud-friendly.

A key outcome of the project is validating the complete data flow from edge to cloud. The live demonstration showed that scan events propagate from the robot to the local gateway, and from there to the AWS cloud where messages are ingested, processed, stored, and visualized. In particular, AWS IoT Core provides the managed IoT entry point, AWS Lambda enables event-driven (serverless) processing and business logic, and DynamoDB stores the inventory state for fast retrieval and dashboard rendering. The Grafana dashboards proved the operational advantage of the approach: operators can observe inventory levels and trends continuously rather than relying on periodic manual checks. In addition, the alerting mechanism (Amazon SNS) demonstrates that the system is not only observational but actionable: low-stock conditions can trigger immediate notifications, reducing the time between a shortage forming and a human responding.

Beyond the live flow, we also explored the “scale mode” concept presented in the project: the same architecture can support a broader product catalog and increased event volume. To demonstrate this without requiring multiple robots or extensive manual scanning, we generated larger streams of scan events using a Python-based simulator. While this does not replace a full multi-robot stress test in a warehouse environment, it validates an important architectural point: the cloud pipeline is designed to remain stable as the number of products and events grows, since ingestion (IoT Core), processing (Lambda), and persistence (DynamoDB) are naturally scalable building blocks.

Security and reliability considerations were addressed as part of the system design. Cloud connectivity is protected using encrypted transport (TLS 1.2) and device authentication based on X.509 certificates in AWS IoT. In addition, IAM-based access control and auditing capabilities (e.g., CloudTrail) provide a foundation for controlled access and traceability. On the operational reliability side, the local gateway plays a central role: it enables local buffering and controlled forwarding to the cloud, which improves robustness in the presence of intermittent connectivity and supports a clearer separation between the robot network and the public cloud.

Together, these choices demonstrate a realistic approach to building a secure and maintainable IoT system using affordable hardware and managed cloud services.

The results also highlight limitations typical for vision-based identification and early-stage prototypes. QR scanning requires line-of-sight and is sensitive to lighting, distance, and motion blur. The evaluation focused on correctness and operational feasibility rather than strict performance metrics, so future work should add quantitative measurements such as end-to-end latency distributions, sustained throughput, and loss/retry behavior under weak WiFi conditions. Additional robustness improvements should include idempotent event IDs and deduplication to prevent double-counting, as well as improved scan reliability (camera placement refinement, brief pause-on-scan policy, or improved decoding pipeline).

Finally, this work establishes a clear roadmap for extending the prototype into a more advanced smart-warehouse system. Immediate extensions include multi-robot support with automated provisioning and topic isolation, improved visualization and analytics for operator decision making, and deeper automation such as reorder recommendations. Longer-term directions include computer-vision-based identification for products without explicit labels and AI-based prediction to forecast shortages and optimize restocking based on historical consumption patterns. Overall, the project demonstrates that an AMR-driven, cloud-connected inventory workflow is feasible, operationally useful, and scalable in design, providing a strong foundation for future development toward a full smart-warehouse solution.

References

- T.-Y. Chai, S. C. Haw, M. Jahangir, K. B. Hoe, L. E. Heng, and M. Vellaisamy. Advancing retail operations: A customizable iot-based smart inventory system. *International Journal of Membrane Science and Technology*, 10(3): 1885–1897, 2023. doi: 10.15379/ijmst.v10i3.1848.
- Z. Li. Review of application of ai in amazon warehouse management. In *Proceedings of ICFTBA 2024 Workshop: Finance’s Role in the Just Transition*, 2024. doi: 10.54254/2754-1169/144/2024.GA18980.
- A. Sagaya Selvaraj and S. Anusha. Rfid enabled smart data analysis in a smart warehouse monitoring system using iot. *Journal of Physics: Conference Series*, 1717:012022, 2021. doi: 10.1088/1742-6596/1717/1/012022.
- S. Yerpude and T. K. Singhal. Smart warehouse with internet of things supported inventory management system. *International Journal of Pure and Applied Mathematics*, 118(24), 2018. URL <http://www.acadpubl.eu/hub/>. Special Issue.